

BEE 4750 Lab 1: Julia and GitHub Basics

Name:

ID:

Due Date

Thursday, 8/28/25, 9:00pm

Setup

The following code should go at the top of most Julia scripts; it will load the local package environment and install any needed packages. You will see this often and shouldn't need to touch it.

```
import Pkg
Pkg.activate(".")
Pkg.instantiate()
```

```
Activating project at `~/lab1-ec8771`
Installing known registries into `~/julia`
  Added `General` registry to `~/julia/registries`
Installed JpegTurbo_jll          v3.1.1+0
Installed Libmount_jll          v2.41.0+0
Installed libfdk_aac_jll        v2.0.4+0
Installed GR_jll                 v0.73.17+0
Installed libdecor_jll          v0.2.2+0
Installed x265_jll               v4.1.0+0
Installed LERC_jll              v4.0.1+0
Installed Preferences            v1.4.3
Installed Opus_jll              v1.5.2+0
Installed RelocatableFolders     v1.0.1
Installed LoggingExtras          v1.1.0
```

Installed Unitful	v1.24.0
Installed Xorg_xkbcomp_jll	v1.4.7+0
Installed Contour	v0.6.3
Installed Xorg_xcb_util_wm_jll	v0.4.2+0
Installed Grisu	v1.0.2
Installed Measures	v0.3.2
Installed ConcurrentUtilities	v2.5.0
Installed Xorg_xcb_util_image_jll	v0.4.1+0
Installed PlotUtils	v1.4.3
Installed GR	v0.73.17
Installed RecipesPipeline	v0.6.12
Installed libsodium_jll	v1.0.21+0
Installed Xorg_libSM_jll	v1.2.6+0
Installed Cairo_jll	v1.18.5+0
Installed DelimitedFiles	v1.9.1
Installed OpenSSL	v1.5.0
Installed HTTP	v1.10.17
Installed Xorg_xcb_util_jll	v0.4.1+0
Installed Statistics	v1.11.1
Installed Fontconfig_jll	v2.16.0+0
Installed Xorg_libXinerama_jll	v1.1.6+0
Installed EpollShim_jll	v0.0.20230411+1
Installed Xorg_libxkbfile_jll	v1.1.3+0
Installed Xorg_libXau_jll	v1.0.13+0
Installed Missings	v1.2.0
Installed Pango_jll	v1.56.3+0
Installed FFMPEG	v0.4.4
Installed PtrArrays	v1.3.0
Installed IrrationalConstants	v0.2.4
Installed Showoff	v1.0.3
Installed JSON	v0.21.4
Installed xkbcommon_jll	v1.9.2+0
Installed Xorg_xcb_util_keysyms_jll	v0.4.1+0
Installed SimpleBufferStream	v1.2.0
Installed XZ_jll	v5.8.1+0
Installed Bzip2_jll	v1.0.9+0
Installed IJulia	v1.29.2
Installed HarfBuzz_jll	v8.5.1+0
Installed PlotThemes	v3.3.0
Installed NaNMath	v1.1.3
Installed LZ0_jll	v2.10.3+0
Installed fzf_jll	v0.61.1+0
Installed FriBidi_jll	v1.0.17+0

Installed UnicodeFun	v0.4.1
Installed SoftGlobalScope	v1.1.0
Installed ColorSchemes	v3.30.0
Installed TranscodingStreams	v0.11.3
Installed ZeroMQ_jll	v4.3.6+0
Installed GLFW_jll	v3.4.0+2
Installed x264_jll	v10164.0.1+0
Installed DataStructures	v0.18.22
Installed StatsAPI	v1.7.1
Installed FreeType2_jll	v2.13.4+0
Installed JLFzf	v0.1.11
Installed CodecZlib	v0.7.8
Installed Compat	v4.18.0
Installed MbedTLS	v1.1.9
Installed libpng_jll	v1.6.50+0
Installed Xorg_libxcb_jll	v1.17.1+0
Installed Colors	v0.13.1
Installed mtdev_jll	v1.1.7+0
Installed libaom_jll	v3.12.1+0
Installed ExceptionUnwrapping	v0.1.11
Installed Scratch	v1.3.0
Installed StatsBase	v0.34.5
Installed Conda	v1.10.2
Installed Zstd_jll	v1.5.7+1
Installed Dbus_jll	v1.16.2+0
Installed ColorTypes	v0.12.1
Installed Xorg_libXext_jll	v1.3.7+0
Installed TensorCore	v0.1.1
Installed eudev_jll	v3.2.14+0
Installed Expat_jll	v2.6.5+0
Installed Xorg_xcb_util_cursor_jll	v0.1.5+0
Installed Libtiff_jll	v4.7.1+0
Installed JLLWrappers	v1.7.1
Installed ZMQ	v1.4.1
Installed Format	v1.3.7
Installed Parsers	v2.8.3
Installed Plots	v1.40.17
Installed Libffi_jll	v3.4.7+0
Installed libinput_jll	v1.28.1+0
Installed ColorVectorSpace	v0.11.0
Installed OrderedCollections	v1.8.1
Installed libevdev_jll	v1.13.4+0
Installed Ogg_jll	v1.3.6+0

Installed Xorg_libXrender_jll	v0.9.12+0
Installed Xorg_libXi_jll	v1.8.3+0
Installed Qt6ShaderTools_jll	v6.8.2+1
Installed Vulkan_Loader_jll	v1.3.243+0
Installed Reexport	v1.2.2
Installed Xorg_libXcursor_jll	v1.2.4+0
Installed Qt6Declarative_jll	v6.8.2+1
Installed MacroTools	v0.5.16
Installed AliasTables	v1.1.3
Installed LogExpFunctions	v0.3.29
Installed Xorg_xcb_util_renderutil_jll	v0.3.10+0
Installed Libuuid_jll	v2.41.0+0
Installed Pixman_jll	v0.44.2+0
Installed Xorg_libICE_jll	v1.1.2+0
Installed Graphite2_jll	v1.3.15+0
Installed libass_jll	v0.17.4+0
Installed StableRNGs	v1.0.3
Installed DocStringExtensions	v0.9.5
Installed Wayland_jll	v1.24.0+0
Installed FFMPEG_jll	v7.1.1+0
Installed Xorg_xkeyboard_config_jll	v2.44.0+0
Installed Xorg_xtrans_jll	v1.6.0+0
Installed BitFlags	v0.1.9
Installed OpenSSL_jll	v3.5.1+0
Installed DataAPI	v1.16.0
Installed Latexify	v0.16.8
Installed LLVMOpenMP_jll	v18.1.8+0
Installed FixedPointNumbers	v0.8.5
Installed Xorg_libXrandr_jll	v1.5.5+0
Installed RecipesBase	v1.3.4
Installed LAME_jll	v3.100.3+0
Installed Xorg_libXfixes_jll	v6.0.1+0
Installed Qt6Wayland_jll	v6.8.2+1
Installed PrecompileTools	v1.2.1
Installed GettextRuntime_jll	v0.22.4+0
Installed Libiconv_jll	v1.18.0+0
Installed Qt6Base_jll	v6.8.2+1
Installed libvorbis_jll	v1.3.8+0
Installed Glib_jll	v2.84.3+0
Installed LaTeXStrings	v1.4.0
Installed VersionParsing	v1.3.0
Installed URIs	v1.6.1
Installed Xorg_libXdmcp_jll	v1.1.6+0

```

Installed Xorg_libX11_jll          v1.8.12+0
Installed Libglvnd_jll            v1.7.1+1
Installed Requires                 v1.3.1
Installed Unzip                   v0.2.0
Installed UnitfulLatexify          v1.7.0
Installed SortingAlgorithms        v1.2.1
Building Conda → ~/.julia/scratchspaces/44cfe95a-1eb2-52ea-b672-e2afdf69b78f/b19db3927f
Building IJulia → ~/.julia/scratchspaces/44cfe95a-1eb2-52ea-b672-e2afdf69b78f/016d15e5d
Precompiling project...
1264.6 ms LaTeXStrings
1140.7 ms StatsAPI
1164.4 ms Contour
1189.5 ms TensorCore
1635.3 ms VersionParsing
1063.5 ms Measures
1327.5 ms Statistics
1940.2 ms Grisu
1437.1 ms OrderedCollections
1163.2 ms Requires
2571.2 ms Format
729.1 ms Reexport
941.5 ms StableRNGs
1239.8 ms Unzip
1254.1 ms DocStringExtensions
754.0 ms PtrArrays
792.6 ms SimpleBufferStream
1319.2 ms URIs
5278.9 ms MacroTools
986.9 ms NaNMath
2361.7 ms IrrationalConstants
1290.3 ms TranscodingStreams
1125.5 ms DelimitedFiles
799.6 ms DataAPI
805.1 ms BitFlags
845.7 ms Scratch
1505.3 ms ConcurrentUtilities
1032.0 ms LoggingExtras
1887.4 ms MbedTLS
1266.9 ms Compat
1417.8 ms Preferences
1600.4 ms SoftGlobalScope
1132.4 ms ExceptionUnwrapping
975.9 ms Showoff

```

1728.9 ms	Statistics → SparseArraysExt
3008.6 ms	UnicodeFun
1441.9 ms	AliasTables
5269.5 ms	FixedPointNumbers
3108.2 ms	LogExpFunctions
2072.6 ms	CodecZlib
1506.1 ms	Missings
1485.8 ms	RelocatableFolders
1807.1 ms	Compat → CompatLinearAlgebraExt
1334.7 ms	PrecompileTools
1596.8 ms	JLLWrappers
3333.4 ms	ColorTypes
8778.9 ms	Latexify
3401.5 ms	RecipesBase
3983.6 ms	DataStructures
1400.7 ms	OpenSSL_jll
997.7 ms	Graphite2_jll
903.0 ms	Libmount_jll
890.2 ms	EpollShim_jll
986.3 ms	LLVMOpenMP_jll
986.5 ms	Bzip2_jll
1017.8 ms	libsodium_jll
943.2 ms	Xorg_libICE_jll
978.0 ms	Xorg_libXau_jll
1077.9 ms	libpng_jll
1119.8 ms	libfdk_aac_jll
1119.9 ms	LAME_jll
1032.5 ms	LERC_jll
1011.4 ms	fzf_jll
1203.6 ms	JpegTurbo_jll
1265.5 ms	XZ_jll
1170.1 ms	Ogg_jll
1057.2 ms	mtdev_jll
1027.5 ms	Xorg_libXdmcp_jll
1134.5 ms	x265_jll
1108.7 ms	x264_jll
1122.2 ms	Zstd_jll
1137.5 ms	libaom_jll
1153.0 ms	Expat_jll
1092.7 ms	LZO_jll
1037.7 ms	Opus_jll
891.4 ms	Xorg_xtrans_jll
876.7 ms	libevdev_jll

1127.9 ms	Libiconv_jll
1139.0 ms	Libffi_jll
1055.4 ms	eudev_jll
1076.0 ms	Libuuid_jll
1217.3 ms	FriBidi_jll
1233.4 ms	ColorTypes → StyledStringsExt
2131.3 ms	Latexify → SparseArraysExt
5530.4 ms	ColorVectorSpace
1844.8 ms	SortingAlgorithms
2945.5 ms	Pixman_jll
2089.3 ms	FreeType2_jll
7217.4 ms	OpenSSL
2065.6 ms	ZeroMQ_jll
11895.5 ms	Colors
1107.3 ms	Xorg_libSM_jll
26885.4 ms	Parsers
1102.9 ms	Xorg_libxcb_jll
2263.4 ms	JLFzf
2293.2 ms	libvorbis_jll
1038.4 ms	DBus_jll
1008.3 ms	Wayland_jll
1120.8 ms	libinput_jll
42934.6 ms	Unitful
3692.6 ms	GettextRuntime_jll
7636.9 ms	Fontconfig_jll
10193.7 ms	Libtiff_jll
6238.9 ms	ZMQ
844.5 ms	Xorg_xcb_util_jll
9932.8 ms	StatsBase
797.0 ms	Xorg_libX11_jll
3183.4 ms	JSON
2124.3 ms	Unitful → PrintfExt
822.3 ms	Xorg_xcb_util_image_jll
830.3 ms	Xorg_xcb_util_keysyms_jll
872.4 ms	Xorg_xcb_util_renderutil_jll
878.4 ms	Xorg_xcb_util_wm_jll
867.7 ms	Xorg_libXrender_jll
3994.5 ms	Glib_jll
916.8 ms	Xorg_libXext_jll
929.2 ms	Xorg_libXfixes_jll
916.0 ms	Xorg_libxkbfile_jll
872.5 ms	Xorg_xcb_util_cursor_jll
2420.2 ms	Conda

```

968.2 ms    Xorg_libXinerama_jll
2465.0 ms   UnitfulLatexify
9501.3 ms   ColorSchemes
948.7 ms    Xorg_libXrandr_jll
970.7 ms    Libglvnd_jll
877.1 ms    Xorg_libXcursor_jll
909.1 ms    Xorg_libXi_jll
880.1 ms    Xorg_xkbcomp_jll
1510.7 ms   Xorg_xkeyboard_config_jll
778.6 ms    xkbcommon_jll
955.5 ms    Vulkan_Loader_jll
8351.4 ms   IJulia
9823.1 ms   Cairo_jll
9655.6 ms   Qt6Base_jll
4848.9 ms   HarfBuzz_jll
19546.4 ms   PlotUtils
6103.0 ms   libass_jll
6209.4 ms   Pango_jll
39175.4 ms   HTTP
696.2 ms    libdecor_jll
847.9 ms    GLFW_jll
10957.9 ms   Qt6ShaderTools_jll
4898.4 ms    PlotThemes
7853.2 ms    RecipesPipeline
3973.8 ms    Qt6Declarative_jll
654.0 ms    Qt6Wayland_jll
9648.8 ms    FFMPEG_jll
3134.7 ms    FFMPEG
4037.5 ms    GR_jll
3484.9 ms    GR
66663.6 ms   Plots
4294.9 ms    Plots → UnitfulExt
4343.6 ms    Plots → IJuliaExt

```

153 dependencies successfully precompiled in 177 seconds. 35 already precompiled.

This next cell loads packages which are required for the rest of the code evaluation. In this case, we only need to load the `Plots.jl` plotting package, but you will see others over the course of the semester (and can add more if desired; just make sure that you’ve [added the new packages to the environment](#)). Standard Julia practice is to load all of the needed packages at the top of the file.

Warning

Loading packages can take a while, especially the first time! Julia tries to precompile all of the packages you're using so repeat use is faster, but this can be quite slow at first.

```
using Plots
```

Introduction

Julia

Julia is an up-and-coming language, originally developed for scientific programming. While learning a new programming language always has its hiccups, the good news is that if you've programmed in a high-level language such as Python or MATLAB, most Julia concepts should look familiar.

If you have not successfully set up Julia, follow the instructions in [Tools Setup](#) and/or ask for help.

You can use other editors for this course, but our recommendation is [Visual Studio Code](#) with the [Julia extension](#), which will make life a *lot* simpler! You should have set this up by following the [Tools Setup](#) instructions, but if not, do so now and/or ask for help.

Jupyter Notebooks

Jupyter notebooks integrate text and equations in Markdown with Julia (or Python, or R) code. To do this, Jupyter notebooks consist of two types of “cells”: code cells and Markdown (text) cells.

Click once on this section of text. A box will appear around this text (and some areas above/below it) - all of that is within this cell.

Markdown is a text markup framework for formatting language that makes things look pretty when viewed across different platforms: web browsers, notebooks, and so forth. Text written in Markdown can also include hyperlinks, LaTeX equations, section headers, and images, among other features (most of [the course website](#) and the lecture notes were all written in Markdown!). [Here is a basic Markdown cheat sheet](#).

What you are looking at right now is the formatted text after the Markdown is processed. To see the raw Markdown, do one of:

- press **Enter** while that cell is selected, or
- double-click on that cell.

A couple of the features you will see in this Markdown cell:

- The `---` command creates a horizontal line. This is also nice for separating sections.
- Backticks (``...``) can be used to format and highlight code, keystrokes, etc.
- The `#` sign is used to create a new section header; two `#` signs (`##`) is used to create a new subsection header; `###` creates a subsubsection, and so on.
- You can create a bulleted list by using the asterisk `*` or a dash `-` and a space.
- You can create regular text by just typing as usual.
- You can create **bold-faced text** by wrapping it with two asterisks on both sides.
- You can create *italicized text* by wrapping it with a single asterisk on both sides.
- To create a new paragraph, you must include a blank line between the old and new paragraphs.

At this point you might be wondering how to turn this cell back into the fully formatted Markdown text instead of the raw Markdown you're probably still looking at. You have a couple of options, depending on your platform, but the most consistent is to type **Shift + Enter** to **execute** the cell (this is also how to run code, but more on that later).

Additionally, you will frequently need to create new cells in your Jupyter notebooks. How you do this will depend on how you interact with the notebook, but try to figure this out now.

One tip is to think carefully about what bits of code should be in the same cell, as you typically only see output from the last command in a cell. For example, compare the following:

```
x = 5
sin(x)
```

-0.9589242746631385

with

```
x = 5
```

5

```
sin(x)
```

-0.9589242746631385

In Julia, you can also suppress the output of a command with a semi-colon:

```
sin(x);
```

which can help if you want to split some code out for clarity or to insert some text prior to it, but don't want to clutter the notebook with its output.

For code cells, to execute the commands within the cell, we also press **Shift+Enter**.

Finally, **make sure that you evaluate all of the code cells in order before submitting**. One bad outcome with notebooks occurs when cells are evaluated out of order, so fixed bugs and edits in previous cells do not get a chance to propagate down. You can do this with the **Run All** command in whichever interface you're using to edit your notebook.

Julia Basics

There are many tutorials and references for Julia, including a [basics overview on the class website](#). Please feel free to reference these as you work through any part of the course.

Formatting Math

It will often be helpful to include nicely-formatted mathematics in a notebook. Markdown accomodates this using LaTeX syntax. A LaTeX cheatsheet is available on the class website, and many other guides exist online.

Below is an example of a formatted equation:

$$x = 5.$$

Looking For Help

There is no shame in using Google, or other resources, for help when programming. There are many, many times when you can't quite get the syntax to work, can't quite figure out the right package or command to use, or are feeling too lazy or overwhelmed (I'm not judging either of those!) to dig through the documentation. Some good resources include:

- [Stack Overflow](#) is a treasure trove of answers;
- The [official Julia forum](#) and the [Julia Subreddit](#) are also very useful.

You are also highly encouraged to post on [Ed Discussion](#), though getting a response might be less immediate. Just be mindful that to get good answers, [you have to help people help you](#), and **make sure to give credit to any resources that were helpful!**

Exercises (3 points)

Use your understanding of Julia syntax and the GitHub workflow to complete the following (hopefully short) exercises. Convert your completed lab assignment to a PDF and submit it to the Gradescope Assignment “Lab 1”.

Remember to:

- Include a (succinct but clear) writeup of the core idea underlying your code, through some combination of equations, text, and algorithms. As you are not required to submit your code, we will not be looking at it in detail, and instead will rely on those writeups to assess whether your approach is correct.
- If using the notebook, evaluate all of your code cells, in order (using a **Run All** command). This will make sure all output is visible and that the code cells were evaluated in the correct order.
- Tag each of the problems when you submit to Gradescope; a 10% penalty will be deducted if this is not done.

Computing a Dot Product

Given two numeric arrays x and y , write a function to compute their dot product if they have equal length, and return an error if not (this is useful for debugging!). Use the following code as a starting point. **Do not use a built-in function.**

```
function dot_product(x, y)
    # if the length of x is equal to length of y
    if (length(x) == length(y)) # insert test condition for equal lengths
        dot_prod = 0 # set base for dot product
        for i in 1:length(x) # iterate through the length of the array
            dot_prod += x[i] * y[i] # compute and return dot product by multiplying x and y
        end
        return dot_prod # then return the dot product
    else
        throw(DimensionMismatch("length of x not equal to length of y"))
    end
end
```

`dot_product` (generic function with 1 method)

Here are some tests to make sure your code works as intended. Tests like these are useful to make sure everything works as intended. One reason to split your code up into functions is that it makes it straightforward to write tests to make sure each piece of your code works, which makes it easier to identify where errors are occurring.

```
dot_product([1 2 3], [4 5 6])
```

32

If you know the value you should get, you can write a more formal test using the `@assert` macro, which is a good way to “automate” checking (since you get an obvious error if the code doesn’t work as desired):

```
@assert dot_product([1 2 3], [4 5 6]) == 32
```

Let’s also make sure we get an error when the dimensions of the two vectors don’t match:

```
dot_product([1 2 3], [4 5])
```

DimensionMismatch: DimensionMismatch: length of x not equal to length of y
DimensionMismatch: length of x not equal to length of y

Stacktrace:

```
[1] dot_product(x::Matrix{Int64}, y::Matrix{Int64})
```

```
@ Main ~/lab1-ec8771/jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X15sZmlsZQ==.jl:10
```

```
[2] top-level scope
```

```
@ ~/lab1-ec8771/jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_X24sZmlsZQ==.jl:1
```

Making a Plot

Write a function to compute the square of an integer x . Evaluate this function for integers between $x = -5$ and $x = 5$ and make a plot of the squared values (you can find a quick guide to making various types of plots [here](#)). Make sure to label your axes.

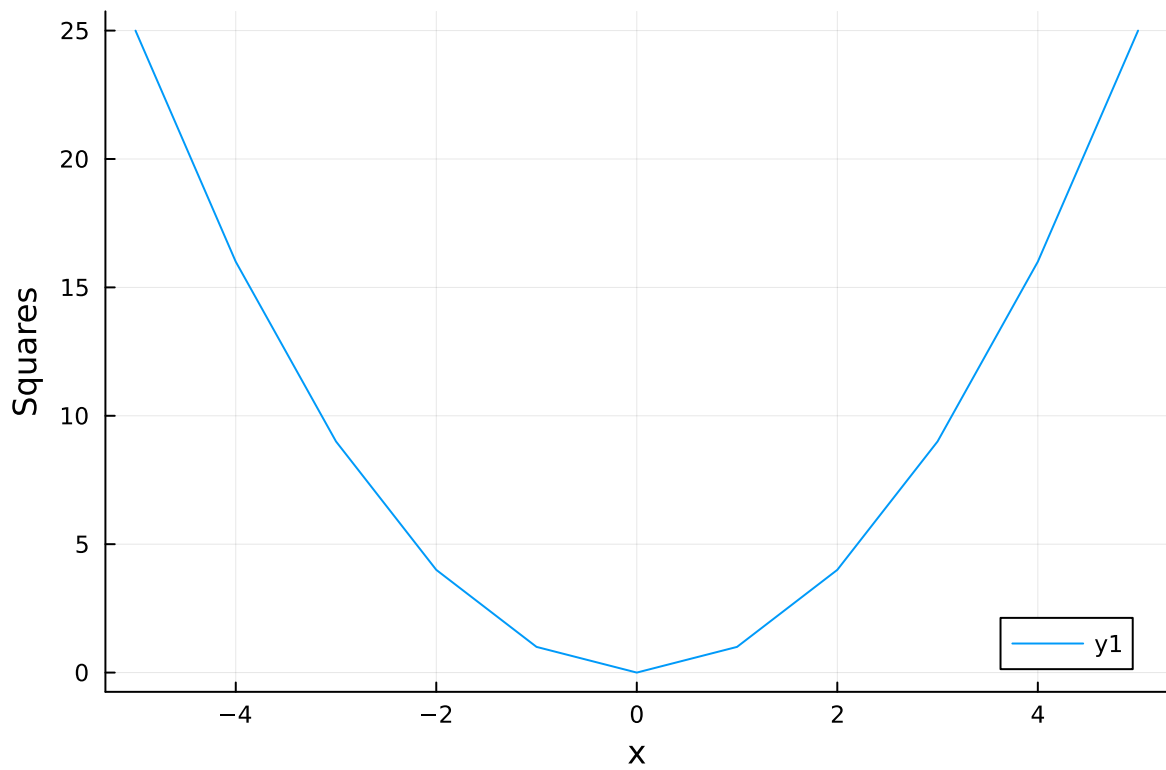
```
# insert your code here
function square(x) # function for squaring
    square = x.*x
    return square
```

```
end

x = range(-5,5) # set x as -5 to 5
y=0 # initialize?
for i in length(x) # for all the values in x
    y = square(x) # square
end

plot(x,y) # plot x and y

# label x and y axes
xlabel!("x")
ylabel!("Squares")
```



Commit and Push Your Changes to GitHub

After completing the previous two exercises, commit your solution file (notebook or otherwise) and push to GitHub. Use an informative commit message which makes it clear what changes you've made. The specific workflow for this will vary depending on how you're writing up your solutions; please search for specifics and ask for help as needed!

Useful Commit Sizes

Ideally, you'd commit whenever you make a "substantial" enough change that you want to lock in, such as writing the core code for a problem or completing a problem, if you're preparing code to be used elsewhere (by yourself or others), or if you want to ask for help. `git` lets you revert changes back to a previous commit, so it's easy to undo changes or updates which broke something that was previously working, so changing too many things at once can make it hard to keep track of what worked when.

But in this case, go ahead and just commit after finishing the problems.

Push the repository with these commits to GitHub and take a screenshot of the repository page (<https://github.com/BEE4750-FA25/<username>/lab01>) which shows the updated repository. Include that screenshot in your submission as the solution to this problem.

Submitting PDF

Important

These submission instructions will not be repeated on future assignments!

Export your writeup as a PDF (this doesn't have to be a notebook if you're struggling to export: you can do your writeup in *e.g.* Microsoft Word) and submit it to the "Lab 1" assignment on Gradescope. **Make sure that you tag pages corresponding to relevant problems to avoid a 10% penalty.**

Printing Code to PDF

You are not required to submit your code when submitting assignments. However, when printing a notebook to PDF (whether you do this locally or via the GitHub Action), long lines will run off the edge of code cells, which may result in comments or code being hidden. If you see this, go back to the notebook and break up long lines into shorter ones (for example, see the comment in the above code cell) to ensure key parts of your results aren't missing.